Variables, Allocation Mémoire RAM, Types Primitifs et Structures Conditionnelles

Thème principal : Fondations de la programmation impérative : modèle mémoire vive (RAM), typage des données, dépassement de capacité (overflow), norme IEEE 754 et évaluation paresseuse dans les branchements conditionnels

Synthèse exécutive : Ce cours pose les fondations fondamentales du développement logiciel en BUT 1 Informatique. L'objectif est de comprendre l'interaction directe entre l'exécution d'un programme et la mémoire vive (RAM) à travers le concept d'abstraction qu'est la variable. La séance démontre la nécessité d'un typage rigoureux pour prévenir le dépassement de capacité (overflow), explicite les pièges de précision des nombres flottants liés à la norme IEEE 754, et détaille le pilotage du flux d'exécution séquentiel par des structures conditionnelles (SI... ALORS... SINON) optimisées par le mécanisme d'évaluation paresseuse (short-circuit).

1. Le Concept de Variable et le Modèle Mémoire Vive (RAM)

Pour qu'un ordinateur exécute un traitement algorithmique ou un programme C++, le processeur doit pouvoir lire et écrire des informations temporaires dans la mémoire vive (RAM). Une variable est une abstraction matérielle haut niveau permettant au développeur de manipuler la RAM sans devoir manipuler directement des adresses hexadécimales physiques complexes (ex: 0x7FFF5FBFF05C). Une variable se caractérise obligatoirement par quatre propriétés indissociables : 1. Le Nom (ou identificateur) : choisi par le développeur dans son code source pour faire référence à l'information. 2. L'Adresse Mémoire : l'emplacement physique réel sur la barrette de RAM attribué par le système d'exploitation. 3. Le Type : indique au compilateur la quantité exacte d'octets à réserver (ex: 4 octets pour un 'int' 32 bits) et la règle de décodage des bits. 4. La Valeur : la suite de 0 et de 1 (motif binaire) actuellement inscrite à cette adresse. Lors d'une opération d'affectation (ex: 'a = 5'), le processeur se rend à l'adresse mémoire associée au nom 'a' et y enregistre le code binaire correspondant à 5. Si une nouvelle affectation survient plus tard ('a = 12'), l'ancienne valeur 5 est définitivement détruite et remplacée par 12. Il n'existe aucun historique des états antérieurs en mémoire RAM.

- Variable : Abstraction matérielle représentant un emplacement réservé dans la mémoire vive (RAM)
- Quatre propriétés indissociables : Nom symbolique, Adresse mémoire physique, Type de donnée, Valeur binaire
- Affectation destructive : L'écriture d'une nouvelle valeur écrase et efface irréversiblement l'ancienne valeur

Cas concret enseignant : *Analogie du casier dans le vestiaire de l'IUT : Chaque casier possède un numéro d'emplacement fixe et immuable (l'adresse mémoire). Le développeur colle une étiquette sur la porte avec un nom symbolique (ex: 'age'). La taille physique du casier détermine son type et sa capacité d'accueil. Placer un sac rouge dans le casier équivaut à affecter la valeur 20. Si vous le remplacez ultérieurement par un sac bleu (valeur 21), le sac rouge disparaît définitivement du casier.*

2. Types Primitifs, Représentation Binaire et Gestion de l'Overflow

La sélection appropriée de la taille d'une variable conditionne la plage de valeurs manipulables. Un octet (8 bits) offre 2^8 = 256 combinaisons possibles. Un entier signé sur 32 bits ('int') s'étend de -2 147 483 648 à +2 147 483 647. Lorsqu'un calcul arithmétique génère une valeur supérieure à la borne maximale d'un entier, le bit de poids fort est tronqué et la valeur 'boucle' brutalement vers la limite minimale négative : c'est le phénomène de dépassement de capacité ou 'overflow'. Concernant les nombres réels à virgule, les ordinateurs utilisent la représentation binaire en virgule flottante (Norme IEEE 754). De même que 1/3 n'a pas d'écriture décimale finie en base 10 (0.3333...), des nombres réels simples comme 0.1 ou 0.2 n'ont pas de représentation binaire exacte en base 2. En conséquence, l'opération '0.1 + 0.2' produit '0.30000000000000004' dans l'ordinateur. Ainsi, il est strictement proscrit de comparer deux flottants avec l'opérateur '==' en partiel. Pour valider l'égalité entre deux flottants 'x' et 'y', il faut vérifier que la valeur absolue de leur différence est inférieure à un seuil d'erreur toléré (epsilon, ex: std::abs(x - y) < 0.00001).

• Types entiers (int, long) : Stockage de valeurs exactes sans virgule, signées ou non signées

• Dépassement de capacité (Overflow) : Bouclage binaire brutal vers la limite négative quand la capacité du type est dépassée

• Types flottants (float, double) : Représentation approchée en base 2 selon la norme IEEE 754

• Interdiction d'égalité stricte (==) : Impossibilité de comparer directement deux float en raison des erreurs d'arrondi

Cas concret enseignant : *Catastrophe de la fusée Ariane 5 (vol 501 en 1996) : Le lanceur s'est autodétruit 37 secondes après son décollage en raison d'un overflow. Un logiciel de bord a tenté de convertir un nombre flottant 64 bits en un entier signé 16 bits. La valeur ayant dépassé 32 767, la conversion a provoqué un dépassement de capacité critique et le plantage du calculateur de bord. Règle d'or de l'enseignant : 'N'utilisez jamais le type float pour gérer des prix ou montants bancaires ! Stockez toujours les sommes financières sous forme d'entiers exprimés en centimes'.*

3. Structures Conditionnelles et Évaluation Paresseuse (Short-Circuit)

Par défaut, un processeur exécute les instructions de manière séquentielle, ligne après ligne de haut en bas. Les structures conditionnelles permettent de dévier ce flux selon la valeur de vérité (VRAI ou FAUX) d'une expression booléenne. L'évaluation paresseuse (ou court-circuit) est un mécanisme d'optimisation fondamental des opérateurs logiques ET (&&) et OU (||) : 1. Dans une expression '(A && B)', si 'A' est FAUX, l'ensemble de l'expression sera obligatoirement FAUX. L'ordinateur n'évalue même pas 'B'. 2. Dans une expression '(A || B)', si 'A' est VRAI, l'ensemble sera obligatoirement VRAI. L'ordinateur n'évalue pas 'B'. Outre l'optimisation du temps de calcul, l'évaluation paresseuse sécurise le code en empêchant des erreurs fatales d'exécution, telles que la tentative d'accès à un pointeur nul ou une division par zéro.

• Exécution séquentielle vs Branchement conditionnel (SI / SINON SI / SINON)

• Opérateurs logiques : ET (&&), OU (||), NON (!)

• Évaluation paresseuse (Short-Circuit) : Interruption de l'évaluation logique dès que le résultat final est mathématiquement garanti

• Prévention des plantages : Sécurisation des accès mémoire et évitement des divisions par zéro

Cas concret enseignant : *Exemple du contrôle d'accès : Dans l'instruction 'SI (utilisateur_existe ET mot_de_passe_correct)', si la condition 'utilisateur_existe' renvoie FAUX, le système n'exécute jamais la recherche du mot de passe en base de données. Cela économise des requêtes BDD coûteuses et évite un plantage grave du programme (de type NullPointer ou Segmentation Fault) si l'objet utilisateur n'existe pas en mémoire.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi 0.1 + 0.2 n'est-il pas exactement égal à 0.3 lorsqu'on l'affiche en programmation ?
Réponse du professeur : Parce que les ordinateurs représentent les nombres réels en binaire selon la norme IEEE 754. Tout comme 1/3 n'a pas d'écriture décimale finie en base 10 (0.3333...), des nombres comme 0.1 et 0.2 n'ont pas de représentation exacte sous forme de somme de puissances négatives de 2. L'addition binaire produit 0.30000000000000004. En C++, on ne compare donc jamais deux float avec '==', mais on teste si leur différence absolue est inférieure à un seuil epsilon : std::abs(a - b) < 0.00001.

Question étudiant : Est-ce qu'une variable déclarée à l'intérieur d'un bloc SI existe toujours après la fin du bloc SINON ?
Réponse du professeur : Non. Une variable déclarée entre les accolades d'un bloc SI possède ce qu'on appelle une 'portée locale' (ou portée de bloc). Dès que l'exécution sort de ce bloc, la mémoire allouée à cette variable est libérée et elle cesse d'exister. Pour qu'une variable soit réutilisable après les structures conditionnelles, elle doit être déclarée en amont, au début de la section VARIABLES du programme.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en C++ permettant de calculer et d'afficher le tarif exact d'un billet de cinéma en fonction de l'âge de l'usager et de son statut d'étudiant, selon la grille suivante : - Tarif plein de base :

10.00 € - Tarif mineur (< 18 ans) : 6.00 € - Tarif étudiant majeur (>= 18 ans et statut étudiant VRAI) : 7.50 € - Tarif senior (>= 65 ans) : 5.00 € Le code doit respecter strictement la norme C++ enseignée à l'IUT d'Anglet (cartouche d'en-tête, tripartition CONSTANTES / VARIABLES / TRAITEMENTS, commentaires de flot de données, accolades Allman).

Démarche de résolution :
Étape 1 : Déclaration des constantes de prix tarifaires et initialisation des variables d'entrée (age, estEtudiant) et de sortie (prixBillet). Étape 2 : Lecture contrôlée des saisies clavier avec commentaires de flot de données. Étape 3 : Ordonnancement rigoureux des tests conditionnels du cas le plus restrictif/avantageux au cas général : - Le premier test 'if (age < 18)' traite tous les mineurs. Un mineur de 16 ans qui est aussi étudiant bascule immédiatement ici et paiera 6.00 € (tarif le plus bas). - Le deuxième test 'else if (age >= 65)' attribue le tarif senior (5.00 €). - Le troisième test 'else if (estEtudiant)' concerne les étudiants majeurs (7.50 €). - Le bloc 'else' final attribue le tarif plein par défaut (10.00 €). Étape 4 : Affichage formaté à deux décimales du tarif attribué.

```
/*
 Nom        : Matéi
 Prénom     : Matéi
 Classe     : BUT1 - Groupe Info
 Matière    : R1.01 - Initiation au Développement
 Date       : 13/11/2026
 Objectif   : Calcul du tarif d'un billet de cinéma selon l'âge et le statut d'étudiant
*/

#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    // ------------------------------------------------------------------
    // CONSTANTES
    // ------------------------------------------------------------------
    const float TARIF_PLEIN = 10.00f;
    const float TARIF_MINEUR = 6.00f;
    const float TARIF_ETUDIANT = 7.50f;
    const float TARIF_SENIOR = 5.00f;

    // ------------------------------------------------------------------
    // VARIABLES
    // ------------------------------------------------------------------
    int age;                // Âge de l'utilisateur en années
    bool estEtudiant;       // Statut d'étudiant (true = oui, false = non)
    float prixBillet;       // Prix final du billet calculé en euros

    // ------------------------------------------------------------------
    // TRAITEMENTS
    // ------------------------------------------------------------------
    // Saisie des données d'entrée
    // (écran) << "Entrez votre age : "
    cout << "Entrez votre age : ";
    // (clavier) >> saisir >> age
    cin >> age;

    // (écran) << "Etes-vous etudiant (1 pour Oui, 0 pour Non) ? : "
    cout << "Etes-vous etudiant (1 pour Oui, 0 pour Non) ? : ";
    // (clavier) >> saisir >> estEtudiant
    cin >> estEtudiant;

    // Détermination du tarif selon la grille conditionnelle
    if (age < 18)
    {
        prixBillet = TARIF_MINEUR;
    }
    else if (age >= 65)
    {
        prixBillet = TARIF_SENIOR;
    }
    else if (estEtudiant)
    {
        prixBillet = TARIF_ETUDIANT;
    }
    else
    {
        prixBillet = TARIF_PLEIN;
    }

    // Affichage du résultat final
    cout << fixed << setprecision(2);
    // (écran) << "Le prix de votre billet est de : " << prixBillet << " euros."
    cout << "Le prix de votre billet est de : " << prixBillet << " euros." << endl;

    return 0;
}
```

Résultat : Validation des cas de test : 1) Utilisateur de 20 ans, estEtudiant = true -> age < 18 (FAUX), age >= 65 (FAUX), estEtudiant (VRAI) -> Tarif attribué : 7.50 € (Conforme). 2) Utilisateur de 16 ans, estEtudiant = true -> age < 18 (VRAI) -> Tarif attribué : 6.00 € (Conforme, la priorité des tests garantit l'application du tarif le plus avantageux).

■ Auto-Évaluation Flash d'Examen

Q : Que se produit-il en mémoire vive lors de l'affectation d'une valeur supérieure à la limite maximale d'un entier signé (ex: attribuer 300 à un int8_t) ?

R : Un dépassement de capacité (overflow) survient : le bit de poids fort est tronqué et la valeur binaire boucle brutalement vers la limite négative minimale (ex: 300 devient -124 dans un entier 8 bits signé).

Q : Dans l'expression conditionnelle 'if (A || B)', quelle est la conséquence du mécanisme d'évaluation paresseuse si l'expression A est évaluée à VRAI ?

R : L'expression B n'est pas évaluée par le processeur, car dès lors que la première condition d'un OU (||) est VRAIE, l'ensemble de l'expression logique est mathématiquement garanti d'être VRAI.

Q : Quelle est la règle impérative enseignée à l'IUT d'Anglet concernant la gestion des montants financiers et prix en programmation ?

R : Ne jamais utiliser de type flottant (float ou double) en raison des imprécisions d'arrondi de la norme IEEE 754, mais représenter systématiquement les sommes d'argent avec des types entiers exprimés en centimes (ex: 1000 centimes pour 10.00 €).